SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct
I _____VINIL KUMAR .S (192511226)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: _____

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge

servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

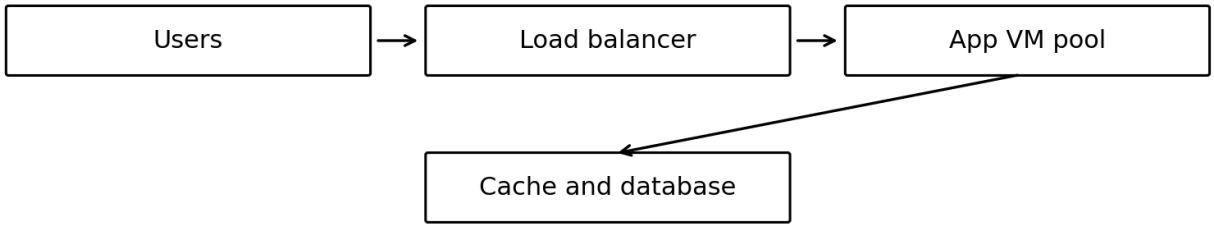
Application Based Questions Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

Question 1 :Auto Scaling and Load Balancing for Seasonal Sales

Handling unpredictable demand

Use an application load balancer with an auto-scaling group of application VMs or container replicas across multiple availability zones. The load balancer distributes incoming requests among healthy instances. Auto-scaling increases capacity when request rate, CPU usage or response latency exceeds a defined target and removes extra instances when demand falls. This improves resource use without maintaining peak capacity throughout the year.



Reducing downtime and improving experience

Health checks remove failed or unready instances from traffic, while instances in other zones continue serving users. Store sessions in a shared cache and orders in a durable database so users are not dependent on one application server. Maintain minimum spare capacity, use warm-up periods and pre-scale before advertised sales because new capacity is not immediate. Cache product images and descriptions, and queue non-urgent work such as confirmation emails. Database capacity must also be planned because adding application servers cannot fix an overloaded database.

Role of monitoring tools

Monitoring collects request counts, response time, error rate, CPU usage and database connections. Metrics trigger scaling policies, while alerts notify operators about failures or slow checkout. Centralized logs help identify failed payments and application errors. Dashboards show whether scaling improves performance; maximum instance limits and budget alerts prevent uncontrolled spending. VM memory limits and resource monitoring help reveal contention between workloads.

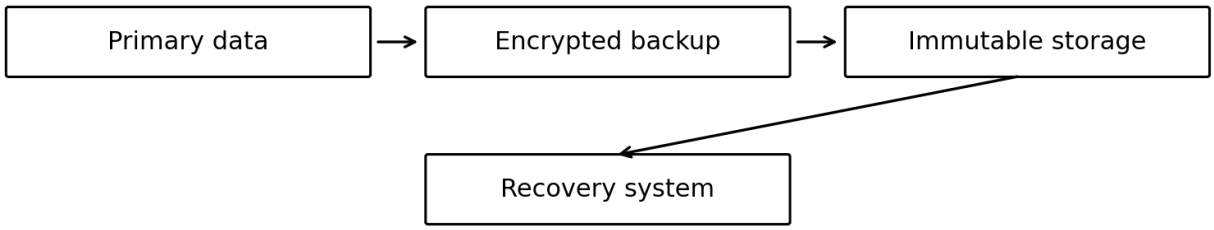
Practical e commerce example

During a festival sale, an online shop may experience a sudden rise from normal traffic to several times its usual demand. The system adds application instances and the load balancer shares requests among them. Cached product pages reduce repeated database reads, and a queue absorbs order-processing bursts. Payment requests use idempotency keys to prevent duplicate charges during retries. After the sale, capacity scales down. This design supports smoother browsing and checkout, provided load testing verifies the application, payment and database limits.

Question 2 : Secure Cloud Backup Storage

Durability and redundancy

Use cloud object storage for large backup files and a managed backup service for databases and VM snapshots. Depending on the selected storage class, the provider stores redundant data across devices and failure domains, often including multiple availability zones. Integrity checks and repair mechanisms help protect against hardware failure. Cross-region backup copies provide additional disaster protection where permitted. Durability concerns retaining data; availability concerns being able to access it when needed.



Encryption and access control

Encrypt backups during transfer using TLS and at rest using managed encryption keys or customer-controlled keys. Restrict backup reading, restoration and deletion using least-privilege roles. Require multifactor authentication for administrators, block public access and log all backup operations. Protect encryption keys separately and ensure they remain available during recovery. Enable versioning and immutable retention because redundant storage alone can reproduce accidental deletion or malicious changes.

Cost advantages and limitations

Cloud storage avoids large initial purchases of storage servers, disks, power and data-centre space. Capacity grows as required, and the organization pays for actual storage use. Lifecycle rules move older backups to cheaper archive classes, while retention policies remove expired copies. However, retrieval fees, transfer charges and minimum retention periods must be included in the cost estimate. Archive storage is suitable for infrequent recovery, but its retrieval delay may not meet a short recovery-time target.

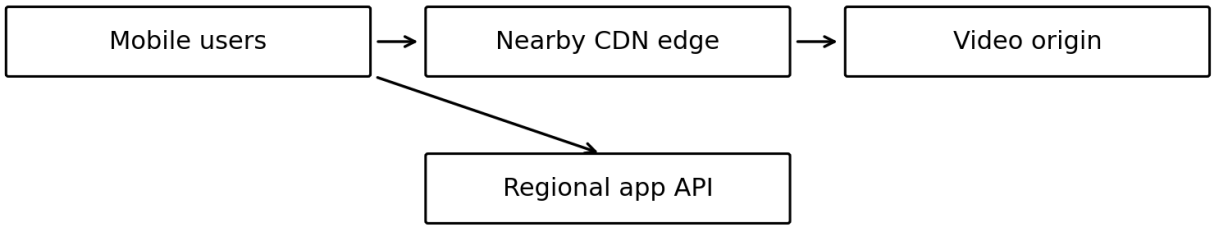
Disaster recovery example

If ransomware damages a company’s local servers, the recovery team isolates the affected network and selects a clean, immutable cloud backup. It restores VM images and database backups into an isolated recovery environment, applies transaction logs where available, validates records and then restores user access. Define the recovery time objective (RTO) and recovery point objective (RPO) in advance. For example, daily backups may lose almost a day of changes; a shorter RPO requires more frequent backups or log capture. Regular restore drills verify that the backup is usable.

Question 3 : Global Mobile Application Delivery

CDN performance and latency

A content delivery network caches application content at edge servers near users. When a user requests a cached image, script or video segment, the nearby edge responds instead of repeatedly contacting a distant origin server. A shorter network path reduces round-trip delay and decreases origin traffic. A cache miss causes the edge to obtain the content from the origin and cache it according to the configured policy.



Edge servers and user experience

Edge caching improves download speed and helps absorb sudden regional demand. Use suitable cache lifetimes and versioned asset names so users receive updated content. Private account information and transaction responses must not enter shared caches. For streaming, encode multiple video bitrates and let the player adapt to changing bandwidth. Edge delivery reduces delay, but it cannot remove latency caused by a slow mobile connection or overloaded application backend.

Global availability through the cloud

Deploy application services on VMs or containers in more than one region, with instances across availability zones. Global traffic management routes users to a nearby healthy regional endpoint, and regional load balancers distribute requests. Health checks and failover redirect traffic when a region becomes unavailable. Replicate required data and plan for replication lag, regional capacity and data-location restrictions. Monitor regional response time, cache hit rate, error rate and video buffering to verify actual user experience.

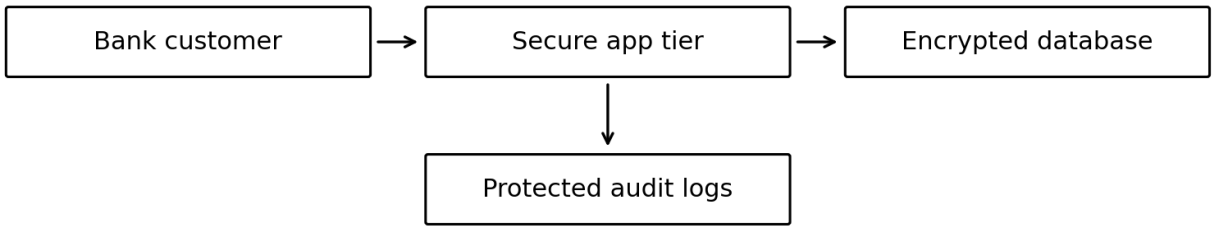
Practical streaming and gaming example

A video-streaming mobile app stores videos at an origin and delivers popular segments through CDN edges in different countries. A viewer receives nearby cached segments, reducing startup time and buffering. The application backend handles authentication and viewing history through healthy regional services. In an online game, a CDN distributes large game updates and textures efficiently, while interactive gameplay uses nearby game servers. Caching alone cannot handle authoritative game state or eliminate real-time multiplayer latency. Combining edge content delivery with regional compute provides a more complete global design.

Question 4 : Financial Cloud Compliance and Data Security

Compliance and auditing support

Choose cloud services whose controls and audit evidence match the institution's applicable regulatory and contractual requirements. Providers offer certifications, independent audit reports, access-management tools and policy controls. The institution must configure and operate its workloads correctly under the shared-responsibility model. Provider certification does not automatically make an application compliant. Maintain a control checklist, document responsibilities and review configurations and access regularly.



Logging monitoring and encryption

Enable centralized audit logs for sign-ins, permission changes, key use, database access and administrative actions. Protect logs against alteration and retain them according to the applicable policy. Security monitoring detects unusual logins, unexpected data transfers and suspicious transaction patterns; alerts support investigation and response. Encrypt connections with TLS and encrypt databases, backups and VM disks at rest. Use least-privilege roles, multifactor authentication, network segmentation and controlled key access. Avoid recording passwords, full card details or other unnecessary sensitive values in logs.

Meeting data sovereignty requirements

Classify sensitive data and identify approved storage and processing locations before deployment. Select permitted regions for databases, replicas, backups and disaster recovery. Enforce region restrictions through organization policies and regularly inspect resources for violations. Check logs, support arrangements, subprocessors and key-management locations where relevant. Data sovereignty concerns the legal jurisdiction over data as well as its physical location. Cross-border replication should occur only when the institution's specific requirements allow it.

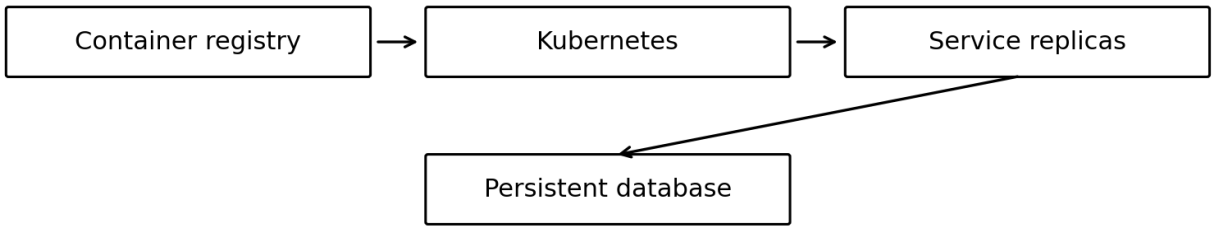
Sensitive transaction example and justification

For an online bank transfer, the user authenticates securely and the request travels through an encrypted connection to a protected application tier. The application authorizes the transfer and commits it to an encrypted database in an approved region. An audit record captures the actor, time, transaction reference and outcome without exposing confidential credentials. Monitoring flags unusual transfer behaviour, and protected backups support recovery. Application VMs or containers remain isolated and patched. This combination supports transaction confidentiality, accountability and recovery while providing evidence for audits.

Question 5 : Containerization and Kubernetes Deployment

Containers compared with virtual machines

A virtual machine runs a complete guest operating system above a hypervisor. A container packages an application and its dependencies while sharing the host operating-system kernel. Containers generally start faster and use less memory and disk space because each one does not require a separate guest OS. VMs provide a stronger isolation boundary and can run different guest kernels. Actual performance depends on workload, storage, networking and configured limits; containers are not automatically faster for every task.



Deployment and scaling with Kubernetes

Kubernetes manages containerized applications through declarative Deployments and Services. It schedules pods onto worker nodes, replaces failed pods and maintains the required replica count. Readiness checks prevent traffic from reaching unready pods, and rolling updates support controlled releases and rollback. Horizontal Pod Autoscaling adjusts replicas using workload metrics; node autoscaling supplies additional worker capacity when needed. Resource requests and limits improve scheduling and reduce contention. A Service provides a stable endpoint even when pod addresses change.

Portability benefits and limitations

The same compatible container image can run on development systems and Kubernetes clusters hosted by different providers. This reduces dependency conflicts and supports consistent releases. Store manifests and infrastructure definitions in version control, and keep configuration and secrets outside the image. Portability still depends on CPU architecture, operating-system compatibility and external services. Provider-specific databases, load balancers and storage classes may require changes. Scan images, run containers without unnecessary privileges and apply network policies because a shared kernel creates security risks.

Microservices application example

An online shopping platform can use separate containers for product catalogue, cart, orders and payments. Kubernetes scales the catalogue service during browsing peaks and the order service during checkout peaks. Services discover one another through internal service names. Keep persistent records in managed databases or persistent volumes rather than a container’s temporary writable layer. Deploy an updated catalogue independently after testing, while payment traffic continues through healthy replicas. This improves release flexibility and resource efficiency, although distributed tracing and reliable service communication are still required.